

Random Access Me...

0720:0113	update	table
6		
0720:0113: 30 048 0		
0720:0114: 31 049 1		
0720:0115: 32 050 2		
0720:0116: 33 051 3		
0720:0117: 34 052 4		
0720:0118: 35 053 5		
0720:0119: 36 054 6		
0720:011A: 37 055 7		
0720:011B: 38 056 8		
0720:011C: 39 057 9		
0720:011D: 0A 010 NEWL		
0720:011E: 0D 013 CRET		
0720:011F: 55 085 U		
0720:0120: 6E 110 n		
0720:0121: 73 115 s		
0720:0122: 6F 111 o		
0720:0123: 72 114 r		
0720:0124: 74 116 t		
0720:0125: 65 101 e		
0720:0126: 64 100 d		
0720:0127: 20 032 SPA		
0720:0128: 61 097 a		
0720:0129: 72 114 r		
0720:012A: 72 114 r		

emulator: noname.exe_

file	math	debug	view	external	virtual devices	virtual drive	help
Load	reload	step back	single step	run	step delay ms: 0		
registers							
H L		F400:0204		F400:0204			
AX	4C 24	F4200: FF 255 RES		BIOS DI			
BX	00 31	F4201: FF 255 RES		INT 021h			
CX	00 00	F4202: CD 205 I		IRET			
DX	01 41	F4203: 21 033 I		ADD [BX + SI], AL			
CS	F400	F4204: CF 207 I		ADD [BX + SI], AL			
IP	0204	F4205: 00 000 NULL		ADD [BX + SI], AL			
SS	0710	F4206: 00 000 NULL		ADD [BX + SI], AL			
SP	00FA	F4207: 00 000 NULL		ADD [BX + SI], AL			
BP	0000	F4208: 00 000 NULL		ADD [BX + SI], AL			
SI	000A	F4209: 00 000 NULL		ADD [BX + SI], AL			
DI	0000	F420A: 00 000 NULL		ADD [BX + SI], AL			
DS	0720	F420B: 00 000 NULL		ADD [BX + SI], AL			
ES	0700	F420C: 00 000 NULL		ADD [BX + SI], AL			
		F420D: 00 000 NULL		ADD [BX + SI], AL			
		F420E: 00 000 NULL		ADD [BX + SI], AL			
		F420F: 00 000 NULL		ADD [BX + SI], AL			
		F4210: 00 000 NULL		ADD [BX + SI], AL			
				...			
screen source reset aux vars debug stack flags							

emulator screen (46x6 chars)

```
Unsorted array: 9 1 0 2 8 5 4 7 3 6
Sorted array: 0 1 2 3 4 5 6 7 8 9
```

clear screen change font 0/16

Mon Oct 6 1:39:55 AM 1% 36% 0%

emulator screen (78x26 chars)

```
enter operand 1: 1
enter operand 2: 0
enter operator {+,-,*,/}: /
division by zero
result: X
want to continue?(y/n):y
enter operand 1: 8
enter operand 2: 3
enter operator {+,-,*,/}: /
result: 2
want to continue?(y/n):y
enter operand 1: 4
enter operand 2: 3
enter operator {+,-,*,/}: +
result: 7
want to continue?(y/n):y
enter operand 1: 9
enter operand 2: 8
enter operator {+,-,*,/}: -
result: 1
want to continue?(y/n):y
enter operand 1: 2
enter operand 2: 4
enter operator {+,-,*,/}: *
result: 8
```

clear screen change font 0/16



COMSATS University Islamabad, Lahore Campus

Department of Computer Engineering

Assignment 1 – FALL 2025

Course Title:	Microprocessor System and Interfacing			Course Code:	CPE 342	Credit Hours:	4(3,1)
Course Instructor:	Mr. Moazzam Ali Sahi			Program Name:	BCE		
Semester:	5 th	Batch:	FA23	Section:	B	Date Given:	October 01, 2025
Submission Date:	October 06, 2025			Maximum Marks:	40		
Name:				Registration Number:			

Important Instructions / Guidelines:

- **Do your own work, PLAGARISM will be graded as ZERO.**
- **No late submission.**
- Provide full screen snapshot of results, clearly indicating time and date from menu.
- Only handwritten assignments will be graded
- Use this as the title page of your assignment.

Question 1: [CLO1-PLO1-C3] [12+4+2+2 Marks]

Apply your understanding of addressing modes and conditional jumps in 8086 assembly to develop a program that sorts 10 unsigned numbers stored in consecutive memory locations (starting at offset (0XXX H) in ascending order. Where XXX is your three-digit registration ID. For e.g, if your registration ID is CIIT/FA23-BCE-001/LHR the starting offset will 0001H.

- Use **looping and conditional jumps (JZ, JNZ, JL, JG)** to implement the sorting algorithm (e.g., Bubble Sort).
- After sorting, display the sorted list on the screen using **INT 21H (function 02H)**.
- Clearly indicate how registers **SI/DI** and addressing modes are used in your program.
- Provide the snapshot of the registers and memory after the execution

Question 2: [CLO1-PLO1-C3] [5+5+5+5 Marks]

Apply your knowledge of arithmetic and program control instructions to implement a menu-driven calculator in 8086 assembly.

- Prompt the user for two numbers and an operation (+, -, *, /).
- Perform the operation using **ADD, SUB, MUL, DIV**.
- Handle division by zero using conditional jumps and display an error message.
- Display the final result using **INT 21H**.
- After displaying the result, prompt the user with an option to rerun the program or exit.

End of Assignment 1